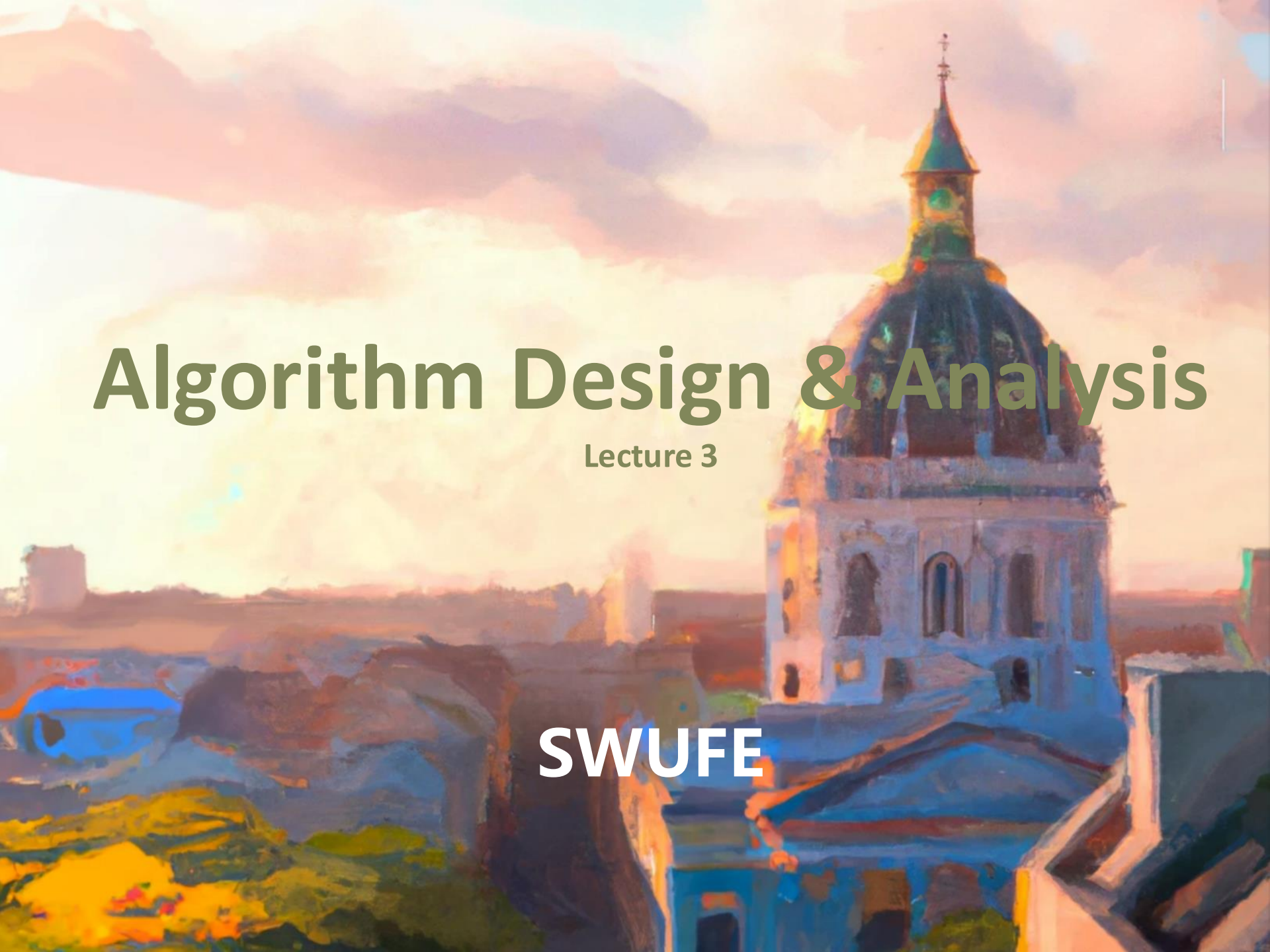


ChatGPT社会化的过程

Introducing ChatGPT

We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible for ChatGPT to answer followup questions, admit its mistakes, challenge incorrect premises, and reject inappropriate requests.

[Try ChatGPT ↗](#)[Read about ChatGPT Plus](#)

An impressionistic painting of a cityscape at sunset. A large, ornate domed church with a green-tiled roof and a spire is the central focus on the right. The sky is filled with soft, warm colors of orange, pink, and purple. The foreground shows rooftops and a body of water reflecting the sky's colors. The overall style is painterly and atmospheric.

Algorithm Design & Analysis

Lecture 3

SWUFE



NOBELPRISET I FYSIK 2024 THE NOBEL PRIZE IN PHYSICS 2024



KUNGL.
VETENSKAPS-
AKADEMIEN

THE ROYAL SWEDISH ACADEMY OF SCIENCES



Photo: Mary Wallham

John J. Hopfield

Princeton University, NJ, USA



Photo: University of Toronto

Geoffrey E. Hinton

University of Toronto, Canada

"för grundläggande upptäckter och uppfinningar som möjliggör maskininlärning med artificiella neuronnätverk"

"for foundational discoveries and inventions that enable machine learning with artificial neural networks"

THE
NOBEL
PRIZE

#NobelPrize

—— 经世济民，孜孜以求 ——



一、序列求和方法

二、递推方程

三、迭代法求解递推方程

四、差消法化简高阶递推方程

五、递归树





数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

数列求和公式

等差、等比数列与调和级数

$$\sum_{k=1}^n a_k = \frac{n(a_1 + a_n)}{2}$$

$$\sum_{k=0}^n aq^k = \frac{a(1 - q^{n+1})}{1 - q}, \quad \sum_{k=0}^n aq^k = \frac{a}{1 - q} (q < 1)$$

$$\sum_{k=1}^n \frac{1}{k} = \ln n + O(1)$$



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

数列求和例子

$$\begin{aligned} \sum_{t=1}^k t 2^{t-1} &= \sum_{t=1}^k t(2^t - 2^{t-1}) \\ &= \sum_{t=1}^k t 2^t - \sum_{t=1}^k t 2^{t-1} = \sum_{t=1}^k t 2^t - \sum_{t=0}^{k-1} (t+1) 2^t \\ &= \sum_{t=1}^k t 2^t - \sum_{t=0}^{k-1} t 2^t - \sum_{t=0}^{k-1} 2^t \\ &= k 2^k - \frac{1(1-2^{k-1+1})}{1-2} \\ &= k 2^k - (2^k - 1) = (k-1)(2^k) + 1 \end{aligned}$$



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

二分检索算法

算法 $\text{BinarySearch}(T, l, r, x)$

输入：数组 T ，下标从 l 到 r ：数 x

输出： j

1. $l \leftarrow 1; r \leftarrow n$

2. *while* $l \leq r$ *do*

3. $m \leftarrow \lfloor (l+r)/2 \rfloor$

4. *if* $T[m] = x$ *then return* m // x 中位元素

5. *else if* $T[m] > x$ *then* $r \leftarrow m-1$

6. *else* $l \leftarrow m+1$

7. *return* 0



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

二分检索算法实例

1	2	3	4	5	6	7
			3.5			

第一次
 $3.5 < 4$

1	2	3	4	5	6	7
	3.5					

第二次
 $3.5 > 2$

1	2	3	4	5	6	7
		3.5				

第三次
 $3.5 > 3$



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

2n+1个输入

假设 $n = 2^k - 1$, 输入有 $2n + 1$ 种:

$$x = T[1]$$

$$x = T[2]$$

$$x = T[3]$$

...

$$x = T[n - 1]$$

$$x = T[n]$$

x 在 T 中

$$x < T[1]$$

$$T[1] < x < T[2]$$

$$T[2] < x < T[3]$$

...

$$T[n - 1] < x < T[n]$$

$$x > T[n]$$

x 不在 T 中



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

比较 t 次的输入个数



比较1次：1个



比较2次：2个



比较3次：4个

对 $t = 1, 2, \dots, k - 1$, 比较 t 次： 2^{t-1} 个
比较 k 次的输入有 $2^{k-1} + n + 1$ 个

总次数：对每个输入乘以次数并求和



数列求和

等差等比
例子

二分检索算法
实例

输入分析

比较分析

平均复杂度

和式估计放大
例子

下界

上界

小结

假设 $n = 2^k - 1$, 各种输入概率相等

$$\begin{aligned} A(n) &= \frac{1}{2n+1} \left[\sum_{t=1}^{k-1} t2^{t-1} + k(2^{k-1} + n + 1) \right] \\ &= \frac{1}{2n+1} \left[\sum_{t=1}^k t2^{t-1} + k(n+1) \right] \\ &= \frac{1}{2n+1} \left[(k-1)2^k + 1 + k(n+1) \right] \\ &= \frac{k2^k - 2k + 1 + k2^k}{2^{k+1} - 1} \approx k = \lfloor \log n \rfloor + \frac{1}{2} \end{aligned}$$



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

估计和式上界的放大法

放大法:

1. 粗糙放大: $\sum_{k=1}^n a_k \leq n a_{\max}$ 放大

2. 等比级数放大: 假设存在常数 $r < 1$, 使得对一切 $k \geq 0$ $a_{k+1} / a_k \leq r$ 成立, 则

$$\sum_{k=0}^n a_k \leq \sum_{k=0}^{\infty} a_0 r^k = a_0 \sum_{k=0}^{\infty} r^k = \frac{a_0}{1-r}$$
 收敛

$a_1 \leq a_0 r, a_2 \leq a_1 r \leq a_0 r^2 \dots$



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

放大法的例子

估计 $\sum_{k=1}^n \frac{k}{3^k}$ 的上界.

解 由

$$a_k = \frac{k}{3^k}, \quad a_{k+1} = \frac{k+1}{3^{k+1}}$$

得

$$\frac{a_{k+1}}{a_k} = \frac{1}{3} \frac{k+1}{k} \leq \frac{2}{3}$$

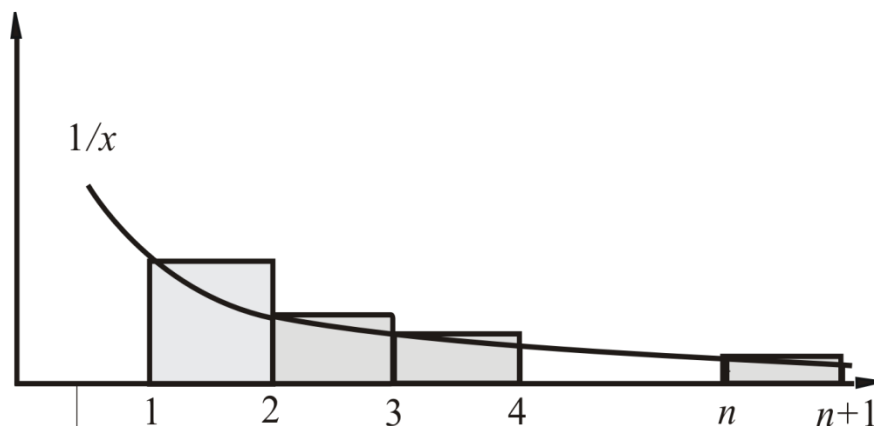
$$\sum_{k=1}^n \frac{k}{3^k} \leq \sum_{k=1}^{\infty} \frac{1}{3} \left(\frac{2}{3}\right)^{k-1} = \frac{1}{3} \frac{1}{1 - \frac{2}{3}} = 1$$

放大的关键步骤

估计和式渐近的界（下界）

估计 $\sum_{k=1}^n \frac{1}{k}$ 的渐近的界.

$$\sum_{k=1}^n \frac{1}{k} \geq \int_1^{n+1} \frac{dx}{x} \\ = \ln(n+1)$$



数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

数列求和

等差等比

例子

二分检索算法

实例

输入分析

比较分析

平均复杂度

和式估计放大

例子

下界

上界

小结

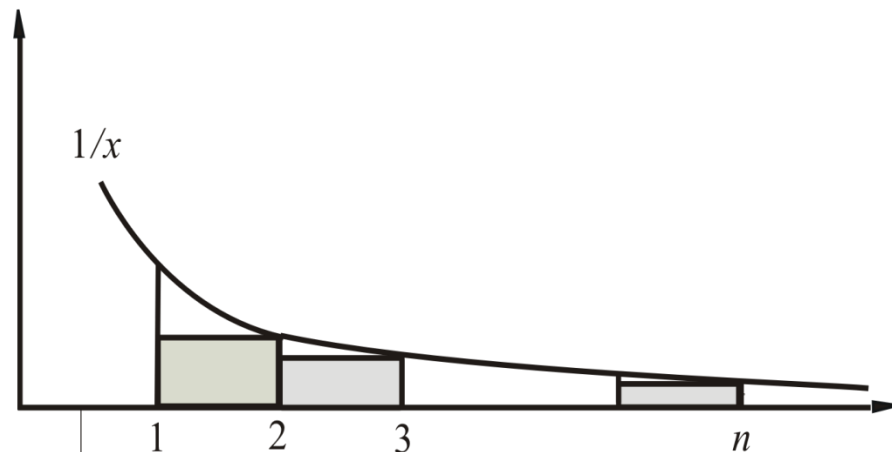
估计和式渐近的界（上界）

估计 $\sum_{k=1}^n \frac{1}{k}$ 的渐近的界.

$$\sum_{k=1}^n \frac{1}{k} = 1 + \sum_{k=2}^n \frac{1}{k}$$

$$\leq 1 + \int_1^n \frac{dx}{x}$$

$$= \ln n + 1$$



$$\sum_{k=1}^n \frac{1}{k} = \Theta(\log n)$$



小结

- 序列求和基本公式
 - 等差数列
 - 等比数列
 - 调和级数
- 估计序列和：
 - 放大法求上届
 - 用积分做和式的渐进的界



一、序列求和方法

二、递推方程

三、迭代法求解递推方程

四、差消法化简高阶递推方程

五、递归树





递推方程

定义

斐波那契例子 实例

Hanoi塔问题 伪码表示 算法分析

插入排序 复杂度分析

小结

递推方程

设序列 $a_0, a_1, \dots, a_n, \dots$, 简记为 $\{a_n\}$, 一个把 a_n 与某些个 $a_i (i < n)$ 联系起来的等式叫做关于序列 $\{a_n\}$ 的递推方程

递推方程的求解:

给定关于序列 $\{a_n\}$ 的递推方程和若干初值, 计算 a_n



递推方程

定义

斐波那契例子

实例

Hanoi塔问题

伪码表示

算法分析

插入排序

复杂度分析

小结

递推方程的例子

• Fibonacci数

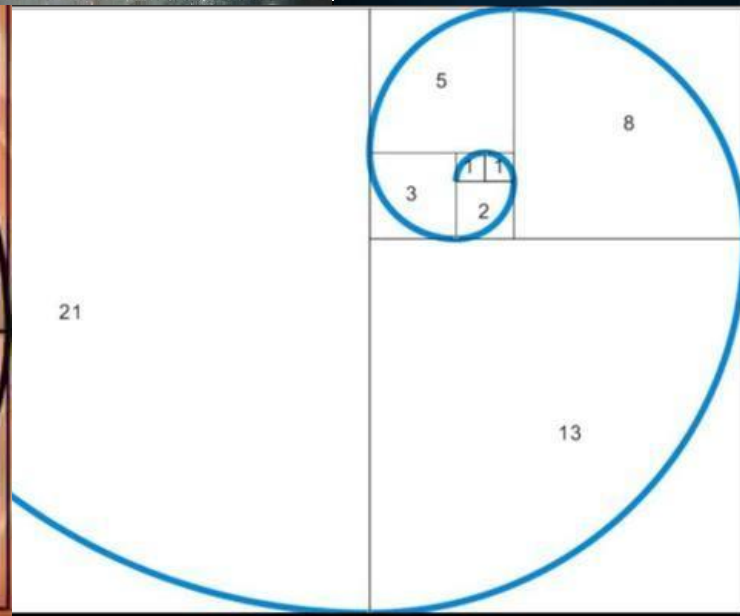
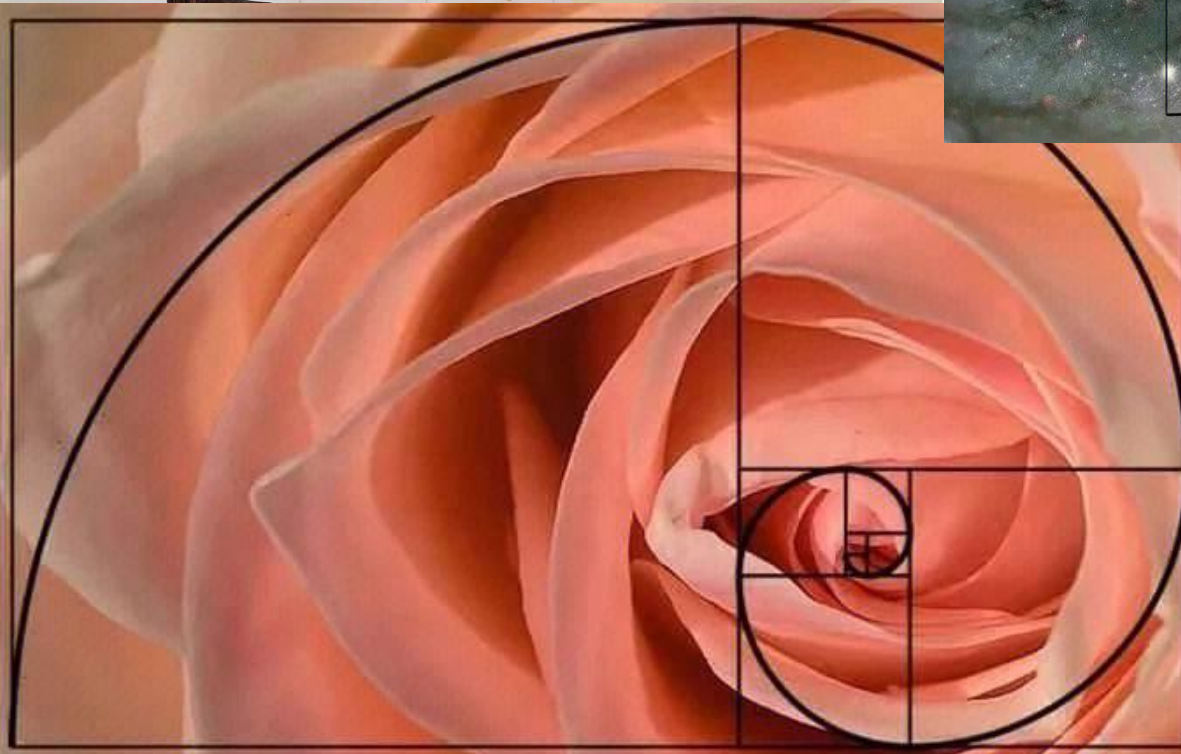
1, 1, 2, 3, 5, 8, 13, 21, 34, 55.....

• 递推方程: $f_n = f_{n-1} + f_{n-2}$
初值: $f_0 = 1, f_1 = 1$



解:

$$f_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1}$$



递推方程

定义

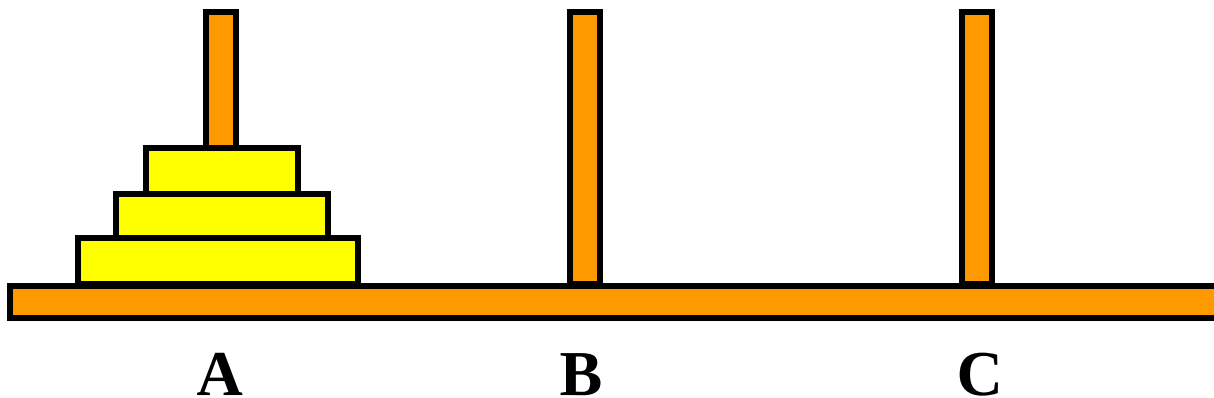
斐波那契例子
实例

Hanoi塔问题
伪码表示
算法分析

插入排序
复杂度分析

小结

Hanoi塔问题



- N个盘子从大到小顺序放在A柱上，要把它们从A移到C，每次移动1个盘子，移动时不允许大盘压在小盘上.设计一种计算移动步骤的方法



递推方程

定义

斐波那契例子
实例

Hanoi塔问题
伪码表示
算法分析

插入排序
复杂度分析

小结

递归算法

算法 Hanoi(A,C,n) // 将A的 n 个盘子按要求移到C

1. if $n=1$ then move (A,C) // 将A的1个盘子移到C

2. else Hanoi(A,B,n-1)

3. move(A,C)

4. Hanoi(B,C,n-1)

设 n 个盘子的移动次数为 $T(n)$

$$T(n) = 2 T(n-1) + 1,$$

$$T(1) = 1,$$



递推方程

定义

斐波那契例子
实例

Hanoi塔问题
伪码表示
算法分析

插入排序
复杂度分析

小结

分析算法

$$T(n) = 2 T(n-1) + 1, \quad T(1) = 1,$$

解: $T(n) = 2^n - 1$

1秒移1个, 64个盘子要多少时间? (5000亿年)
一秒千万亿次, 需要4个多小时



有没有别的更好的算法?

没有! 这是一个难解的问题, 不存在多项式时间的算法!



递推方程

定义

斐波那契例子
实例

Hanoi塔问题
伪码表示
算法分析

插入排序
复杂度分析

小结

插入排序

算法 InsertSort(A, n) // A 为 n 个数的数组

1. for $j \leftarrow 2$ to n do

2. $x \leftarrow A[j]$

3. $i \leftarrow j - 1$ // 行3到行7把 $A[j]$ 插入 $A[1..j-1]$ 之中

4. while $i > 0$ and $x < A[i]$ do

5. $A[i+1] \leftarrow A[i]$

6. $i \leftarrow i - 1$

7. $A[i+1] \leftarrow x$



最坏情况下的复杂度

递推方程

定义

斐波那契例子
实例

Hanoi塔问题
伪码表示
算法分析

插入排序
复杂度分析

小结

插入排序：

设基本运算是元素比较，对规模为 n 的输入最坏情况下的时间复杂度 $W(n)$

$$\begin{cases} W(n) = W(n-1) + n - 1 \\ W(1) = 0 \end{cases}$$

解为：

$$W(n) = n(n-1)/2$$



小结

- 递推方程的定义与初值
- 递推方程与算法时间复杂度的关系：
Hanoi塔的递归算法
插入排序的迭代算法



一、序列求和方法

二、递推方程

三、迭代法求解递推方程

四、差消法化简高阶递推方程

五、递归树





迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

迭代法求解递推方程

迭代法：

1. 不断用递推方程的右部替换左部
2. 每次替换，随着 n 的降低在和式中多出一项
3. 直到出现初值停止迭代
4. 将初值代入并对和式求和
5. 可用数学归纳法验证解的正确性



迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

Hanoi塔迭代求解实例

$$\begin{aligned} T(n) &= 2 T(n-1) + 1 \\ T(1) &= 1 \end{aligned}$$

$$T(n) = 2 T(n-1) + 1$$

$$= 2 [2T(n-2) + 1] + 1$$

$$= 2^2 T(n-2) + 2 + 1$$

$$= \dots$$

$$= 2^{n-1} T(1) + 2^{n-2} + 2^{n-3} + \dots + 2 + 1$$

$$= 2^{n-1} + 2^{n-1} - 1$$

代入
初值

$$= 2^n - 1$$



迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

插入排序迭代求解实例

$$W(n) = W(n-1) + n - 1$$

$$= [W(n-2) + n - 2] + n - 1$$

$$= W(n-2) + (n-2) + (n-1)$$

$$= [W(n-3) + n - 3] + (n-2) + (n-1)$$

$$= \dots$$

$$= W(1) + 1 + 2 + \dots + (n-2) + (n-1)$$

$$= 1 + 2 + \dots + (n-2) + (n-1)$$

$$= n(n-1)/2$$

$$\begin{cases} W(n) = W(n-1) + n - 1 \\ W(1) = 0 \end{cases}$$



迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

换元迭代

有的时候递推迭代非常困难

- 将对 n 的递推式换成对其他变元 k 的递推式
- 对 k 直接迭代
- 将解（关于 k 的函数）转换成关于 n 的函数



迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

二分归并排序算法

MergeSort(A, p, r)

输入：数组 $A[p \dots r]$

输出：按递增顺序排序的数组 A

1. if $p < r$
2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$
3. MergeSort(A, p, q)
4. MergeSort($A, q+1, r$)
5. Merge(A, p, q, r)



迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

换元求解

假设 $n=2^k$ ，递推方程如下：

$$\begin{cases} W(n) = 2W(n/2) + n - 1 \\ W(1) = 0 \end{cases}$$

换元：

$$\begin{aligned} W(2^k) &= 2W(2^{k-1}) + 2^k - 1 \\ W(0) &= 0 \end{aligned}$$



迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

迭代求解过程

$$W(2^k) = 2W(2^{k-1}) + 2^k - 1$$
$$W(0) = 0$$

换元对
k迭代

$$\begin{aligned} W(n) &= 2W(2^{k-1}) + 2^k - 1 \\ &= 2[2W(2^{k-2}) + 2^{k-1} - 1] + 2^k - 1 \\ &= 2^2W(2^{k-2}) + 2^k - 2 + 2^k - 1 \\ &= 2^2[2W(2^{k-3}) + 2^{k-2} - 1] + 2^k - 2 + 2^k - 1 = \dots \\ &= 2^k W(1) + k2^k - (2^{k-1} + 2^{k-2} + \dots + 2 + 1) \\ &= k2^k - 2^k + 1 \\ &= n \log n - n + 1 \end{aligned}$$



迭代法求解

定义

Hanoi实例

插入排序实例

换元法

二分归并换元
算法

换元求解

迭代过程

归纳验证

小结

解的正确性-归纳验证

证明：下述递推方程的解是 $W(n) = n(n - 1)/2$

$$W(n) = W(n - 1) + n - 1$$

$$W(1) = 0$$

方法：数学归纳

$$\text{证： } n = 1, W(1) = 1 \times (1 - 1)/2 = 0$$

假设对于 n ，解满足方程，则

$$W(n + 1)$$

$$= W(n) + n = n(n - 1)/2 + n$$

$$= n[(n - 1)/2 + 1] = n(n + 1)/2$$



小结

- 迭代法求解递推方程
- 直接迭代，代入初值，然后求和
- 对递推方程和初值进行换元，然后求和，求和后进行相反换元得到原始递推方程的解
- 验证方法—数学归纳法



一、序列求和方法

二、递推方程

三、迭代法求解递推方程

四、差消法化简高阶递推方程

五、递归树



差消化简

快速排序实例

输入情况

工作量和

平均工作量

差消化简1

差消化简2

差消化简3 例子

小结

差消化简高阶递推方程

快速排序（迭代法无法化解高阶方程）

- 假设 $A[p..r]$ 的元素彼此不等
- 以首元素 $A[1]$ 对数组 $A[p..r]$ 划分，使得：
 - 小于 x 的元素放在 $A[p..q-1]$
 - 大于 x 的元素放在 $A[q+1..r]$
- 递归对 $A[p..q-1]$ 和 $A[q+1..r]$ 排序

工作量：子问题工作量+划分工作量

输入情况

- Pivot的实际位置有 n 种可能的情况

x排好序位置	子问题1规模	子问题2规模
1	0	$n-1$
2	1	$n-2$
3	2	$n-3$
...	...	...
$n-1$	$n-2$	1
n	$n-1$	0

额外工作量：对每个输入，划分的比较次数都是 $n-1$



差消化简

快速排序实例

输入情况

工作量和

平均工作量

差消化简1

差消化简2

差消化简3
例子

小结

工作量总和

首元素在第一个位置 $T(0)+T(n-1)+n-1$

首元素在第二个位置 $T(1)+T(n-2)+n-1$

$T(2)+T(n-3)+n-1$

...

$+ T(n-1)+T(0)+n-1$

$2[T(1)+...T(n-1)]+n(n-1)$



差消化简

快速排序实例

输入情况

工作量和

平均工作量

差消化简1

差消化简2

差消化简3

例子

小结

快速排序平均工作量总和

假设首元素排好序在每个位置是等概率的

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(n), n \geq 2$$

$$T(1) = 0$$

全部历史的递推方程

对于高阶方程应该先简化，然后迭代



差消化简

快速排序实例

输入情况

工作量和

平均工作量

差消化简1

差消化简2

差消化简3
例子

小结

差消化简-1

利用两个方程相减，将右边的项尽可能消去，以达到降阶的目的

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + cn$$

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + c(n-1)^2$$



差消化简

快速排序实例

输入情况

工作量和

平均工作量

差消化简1

差消化简2

差消化简3

例子

小结

差消化简-2

$$\begin{cases} nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2 \\ (n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + c(n-1)^2 \end{cases}$$

上述两式相减

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + cn^2 - c(n-1)^2$$

化简得到

$$nT(n) = (n+1)T(n-1) + c_1n$$

变形得到

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c_1n}{n(n+1)} = \frac{T(n-1)}{n} + \frac{c_1}{(n+1)}$$



差消化简

快速排序实例

输入情况

工作量和

平均工作量

差消化简1

差消化简2

差消化简3

小结

差消化简-3

变形并迭代得到

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c_1}{(n+1)}$$

= ...

$$= c_1 \left[\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} \right] + \frac{T(1)}{2}$$

代入
初值

$$= c_1 \left[\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} \right]$$

调和级数

= $\Theta(\log n)$ 上述式中括号内为调和级数之和

$$T(n) = \Theta(n \log n)$$



小结

- 对于高阶递推方程先用差消法化简为一阶方程
- 迭代求解



一、序列求和方法

二、递推方程

三、迭代法求解递推方程

四、差消法化简高阶递推方程

五、递归树





递归树

递归树的概念

- 递归树是迭代计算的模型
- 递归树的生成过程与迭代过程一致
- 递归树上所有的项恰好是迭代之后产生和式中的项
- 对递归树上的项求和后就是迭代后方程的解

递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

实例分析2

树上量求和

递归树应用

小结

递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

实例分析2

树上量求和

递归树应用

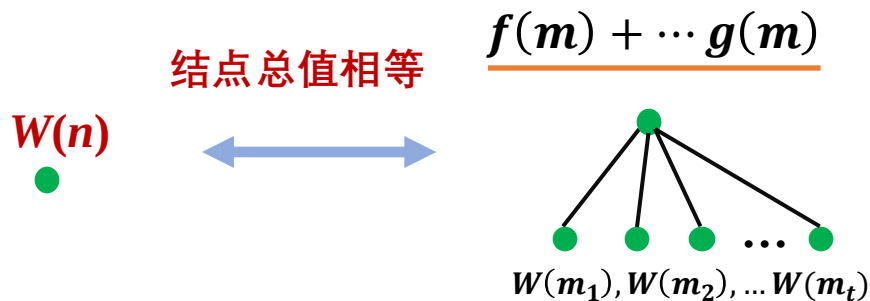
小结

迭代在递归树中的表示

- 如果递归树上某结点标记为 $W(m)$

$$W(m) = W(m_1) + \cdots + W(m_t) + \underline{f(m) + \cdots g(m)}, \quad m_1, \dots, m_t < m$$

其中 $W(m_1), \dots, W(m_t)$ 称为函数项。

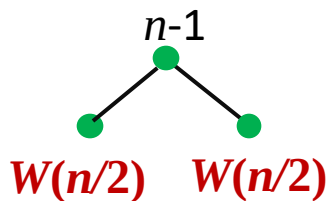




二层子树的例子

二分归并排序

$$W(n) = 2W(n/2) + n - 1$$



递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

实例分析2

树上量求和

递归树应用

小结



递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

实例分析2

树上量求和

递归树应用

小结

递归树生成规则

- 初始，递归树只有根结点，其值为 $W(n)$
- 不断继续下述过程：
 - 将函数项叶节点的迭代式 $W(n)$ 表示成二层子树
 - 用该子树替换该叶结点
- 继续递归树的生成，直到树中无函数项（只有初值）为止

递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

实例分析2

树上量求和

递归树应用

小结

递归树生成实例-1

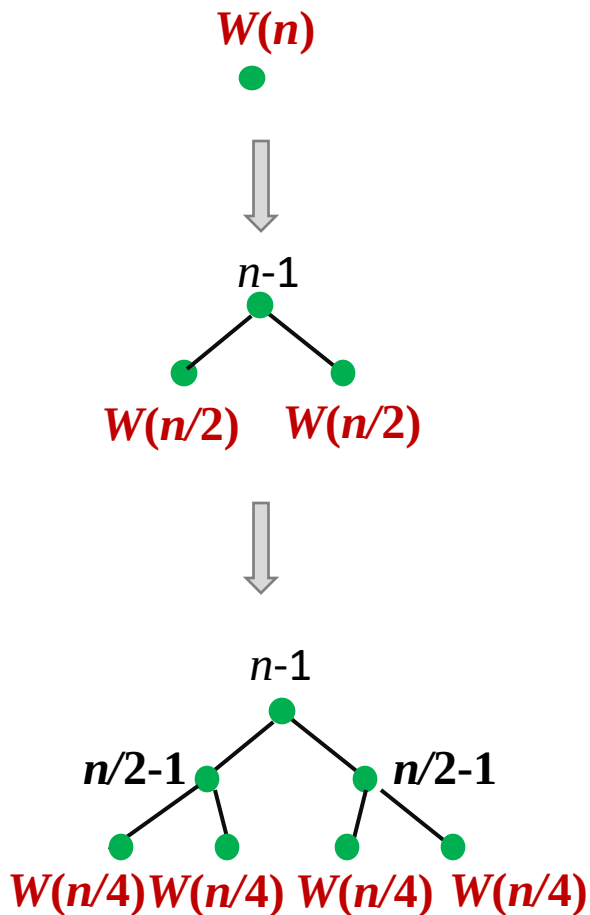
$$W(n) = 2W(n/2) + n - 1$$

初始

第一步迭代

第二步迭代

直到叶结点出现初值





递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

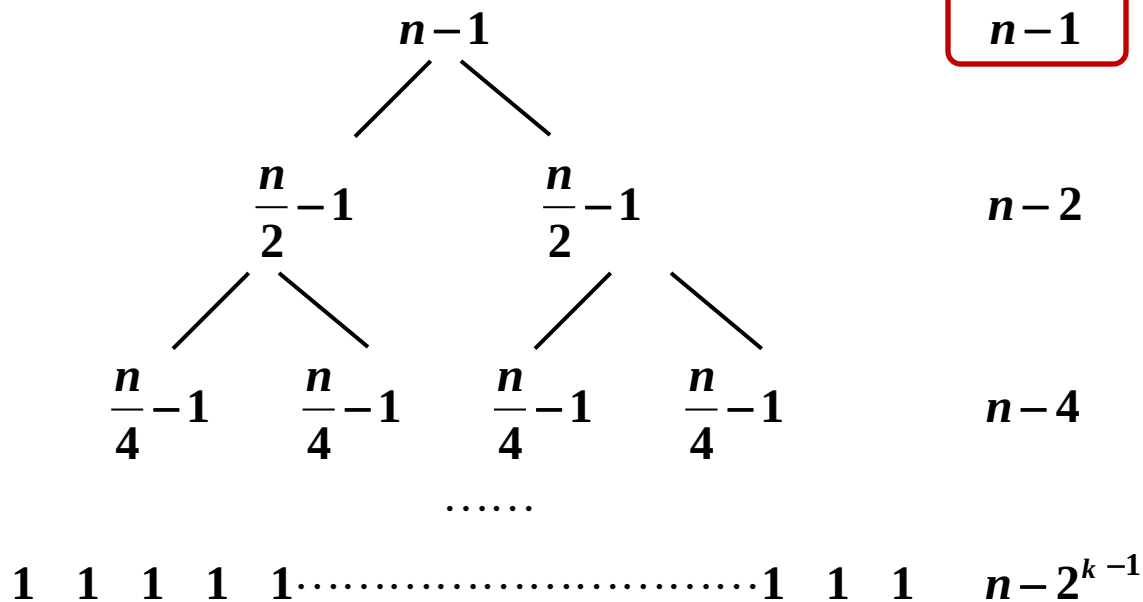
实例分析2

树上量求和

递归树应用

小结

递归树生成实例-2





递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

实例分析2

树上量求和

递归树应用

小结

递归树上的量求和

$$W(n) = 2W(n/2) + n - 1, \quad n = 2^k$$

$$W(1) = 0$$

$$W(n) = n - 1 + n - 2 + \dots + n - 2^{k-1}$$

$$= kn - (2^k - 1)$$

$$= n \log n - n + 1$$



递归树

概念

迭代递归表示

二层递归例子

生成规则

实例分析1

实例分析2

树上量求和

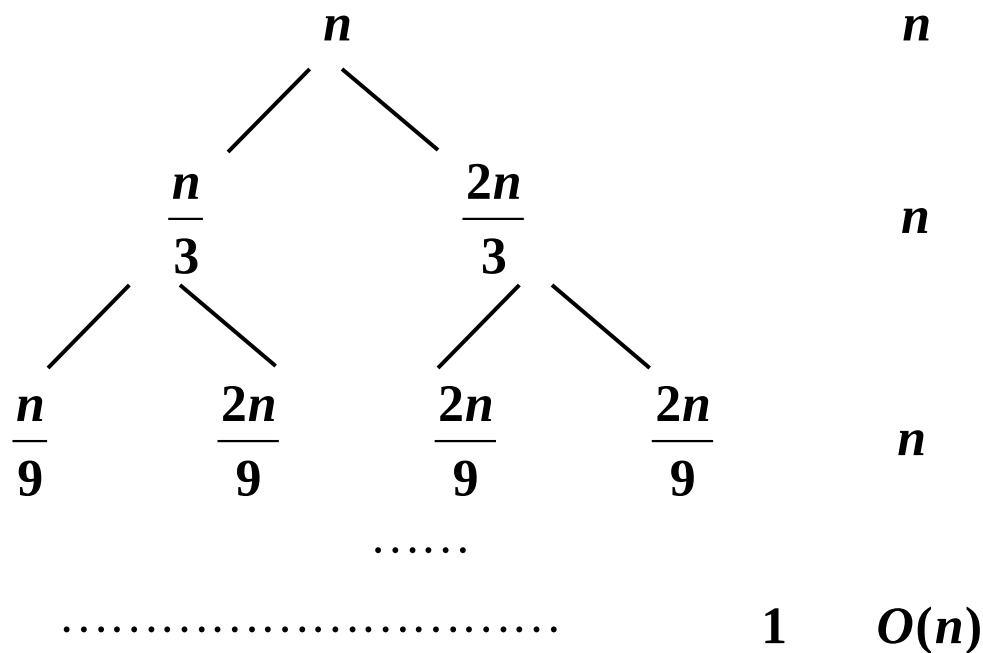
递归树应用

小结

递归树应用实例

层数如何确定?

求解: $T(n) = T(n/3) + T(2n/3) + n$



层数 k : $n(2/3)^k = 1 \Rightarrow (3/2)^k = n \Rightarrow k = O(\log_{3/2} n)$

$T(n) = O(n \log n)$



小结

- 递归树是迭代的图形表述
- 递归树的生成规则
- 如何利用递归树求解递归方程



主定理

求解递推方程

$$T(n) = aT(n/b) + f(n)$$

a : 规约后的子问题个数

n/b : 规约后子问题的规模

$f(n)$: 规约过程及组合子问题的解的工作量

二分检索: $T(n) = T(n/2) + 1$

二分归并排序: $T(n) = 2T(n/2) + n - 1$

主定理

定理1.6 设 $a \geq 1, b > 1$ 为常数, $f(n)$ 为函数, $T(n)$ 为非负整数, 且

$$T(n) = aT(n/b) + f(n)$$

则有以下结果:

1. 若 $f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0$, 那么

$$T(n) = \Theta(n^{\log_b a})$$

存在
上界

2. 若 $f(n) = \Theta(n^{\log_b a})$, 那么

$$T(n) = \Theta(n^{\log_b a} \log n)$$

等阶

3. 若 $f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0$, 且对某个常数 $c < 1$ 和所有充分大的 n 有 $a f(n/b) \leq c f(n)$, 那么

$$T(n) = \Theta(f(n))$$

存在
下界



主定理的证明

不妨设 $n=b^k$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$= a\left[aT\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right)\right] + f(n) = a^2T\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n)$$

= ...

$$= a^kT\left(\frac{n}{b^k}\right) + a^{k-1}f\left(\frac{n}{b^{k-1}}\right) + \dots + af\left(\frac{n}{b}\right) + f(n)$$

$$= a^kT(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \quad T(1) = c_1$$

$$a^{\log_b n} = n^{\log_b a}$$



$$\begin{aligned}T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) & f(n) &= O(n^{\log_b a - \varepsilon}) \\&= c_1 n^{\log_b a} + O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) \\&= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a - \varepsilon})^j}\right) \\&= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j\right) \\&= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1}\right) \\&= c_1 n^{\log_b a} + O(n^{\log_b a - \varepsilon} n^\varepsilon) = \Theta(n^{\log_b a})\end{aligned}$$



一、序列求和方法

二、递推方程

二、迭代法求解递推方程

三、差消法化简高阶递推方程

四、递归树





作业：截至时间**3月23日**前上传**QQ作业**

定理1.1 设 f 和 g 是定义域为自然数集合的函数.

(1) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在且等于某个常数 $c > 0$, 那么

$$f(n) = \Theta(g(n)).$$

(2) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, 那么 $f(n) = o(g(n))$.

(3) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$, 那么 $f(n) = \omega(g(n))$.